



**UNIVERSITY
OF LONDON**

University of London BSc Computer Science

CM3045: 3D graphics & Animation

Student Name: Tan Liang Ting

Student Registration Number (SRN): 230656747

Overview

This project implements a small interactive volleyball prototype scene in Unity with three core tasks: (1) a volleyball net that is visually readable while remaining physically collidable, (2) a physics-driven player controller with grounded jumping, and (3) a third-person orbit camera with mouse look, smoothing, and optional camera collision. The emphasis is on applying real-time graphics and animation principles in a way that produces stable interaction and predictable aesthetics suitable for gameplay prototyping.

Task 1: Net rendering and collision

The aim is to create a net that communicates the expected volleyball “mesh” appearance without expensive geometry, while behaving as a solid barrier the player can collide with. (See figure 1)

Texture mapping and tiling. A 2D texture $T(u, v)$ is mapped onto a mesh using UV coordinates so the net squares stay visible even though the mesh itself is simple. For each rendered fragment, the shader samples the base map at (u, v) to get colour C . Tiling scales UVs before sampling:

$$u' = u \cdot s_x, v' = v \cdot s_y$$

In my material `mat_net`, the base texture is tiled at $(x = 10, y = 3)$, meaning the net pattern repeats 10 times horizontally and 3 times vertically across the mesh surface.

Collider-based blocking. To make the net solid, I added a collider, so Unity’s physics system handles collisions automatically when the player hits it.

Implementation and evaluation

In my scene, the net uses a transparent material (`mat_net`) and I increased the texture tiling (BaseMap scale) so the net pattern stays readable. I used a MeshCollider on the net so the player rigidbody cannot pass through it. I chose this approach because it is efficient and easy to control: the mesh stays low-poly, and most of the detail comes from the texture. A more realistic option would be modelling strands or using a more advanced shader for thickness and lighting, but that would cost more performance and time for a prototype.

Task 2: Rigidbody player movement and grounded jumping

The aim is to create a controllable player object that can move with WASD, jump consistently, and collide properly with the ground and the net. (See figure 2)

Rigidbody dynamics. I used a Rigidbody on the player, so movement and collisions are handled by the physics engine. Planar movement is implemented by moving the current

velocity toward a target velocity each physics step. In Unity 6, I used `Rigidbody.linearVelocity` to manage velocity directly.

Camera-relative movement. I mapped WASD input using the camera's forward directions. This makes movement stay intuitive even when the camera rotates.

Jump impulse. The jump uses an impulse force (`AddForce(..., ForceMode.Impulse)`) so the upward velocity change happens immediately, which suits a jump action.

Ground check. I used `Physics.CheckSphere` at a `GroundCheck` point under the player to decide if jumping is allowed. This prevents double jumps and makes jumping more reliable.

Implementation and evaluation

The `PlayerController` reads the input axes, converts them into a camera-relative direction, and updates `linearVelocity` inside `FixedUpdate` so it stays consistent with physics timing. Jumping only triggers when the ground check returns true, which prevents jumping in mid-air and improves control consistency. I used a rigidbody approach because it naturally interacts with colliders and produces believable contact behaviour. A `CharacterController` would be another option, but it is kinematic, so it often needs extra handling to get similar physics interactions.

Task 3: Third-person orbit camera with smoothing and optional collision

The aim is to provide a stable third-person view that rotates around the player using mouse input, supports zoom, and avoids harsh snapping. (See figure 3 and 4)

Yaw/pitch orbit with clamping. Mouse movement updates yaw (horizontal rotation) and pitch (vertical rotation). I clamp pitch using `Mathf.Clamp` so the camera cannot flip over or go to extreme angles.

Quaternion rotation. I use `Quaternion.Euler` to build the orbit rotation from yaw/pitch and apply it to an offset vector to position the camera around the player. Quaternions are commonly used in Unity for stable 3D rotations.

Smoothing. I smooth camera movement using `Vector3.Lerp` so it blends toward the desired orbit position instead of snapping instantly.

Implementation and evaluation

The camera script calculates the orbit position each frame in `LateUpdate` so it follows the player movement updates. Pitch is clamped, and the camera position is interpolated to reduce jitter. I used this approach because it is simple, easy to tune, and the maths is straightforward to explain in a report. Cinemachine would be the professional alternative, but writing the orbit camera myself makes the underlying method clear for assessment.

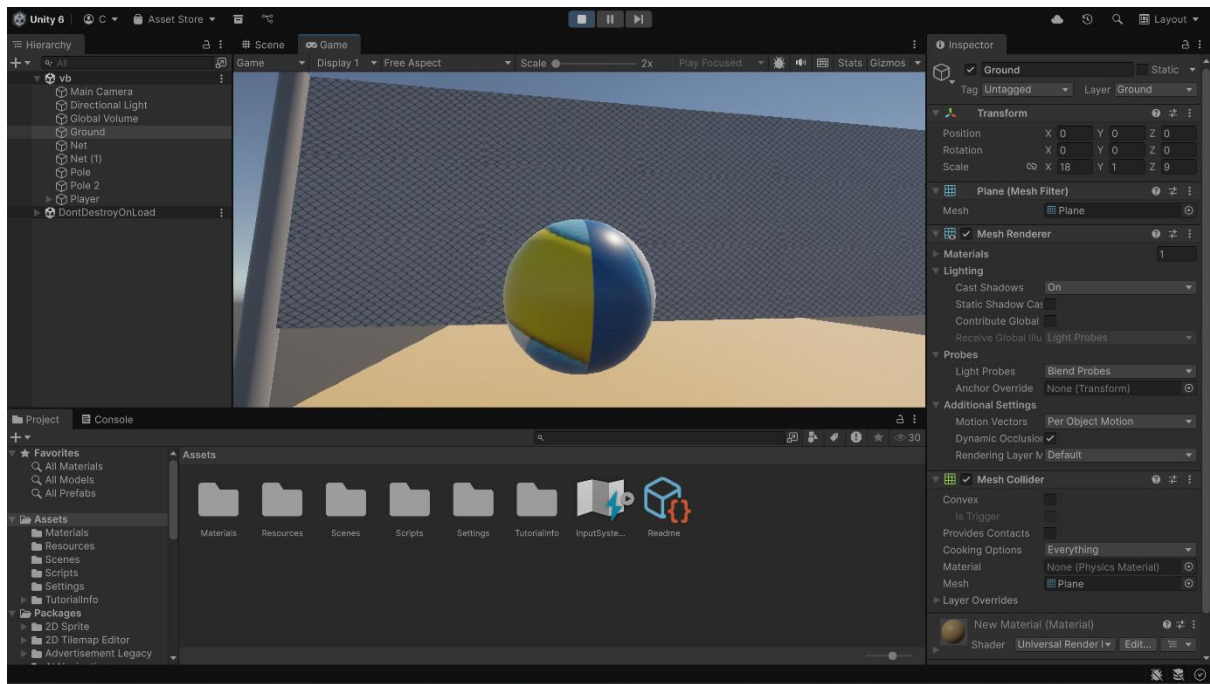


Figure 1: Volleyball colliding with the net

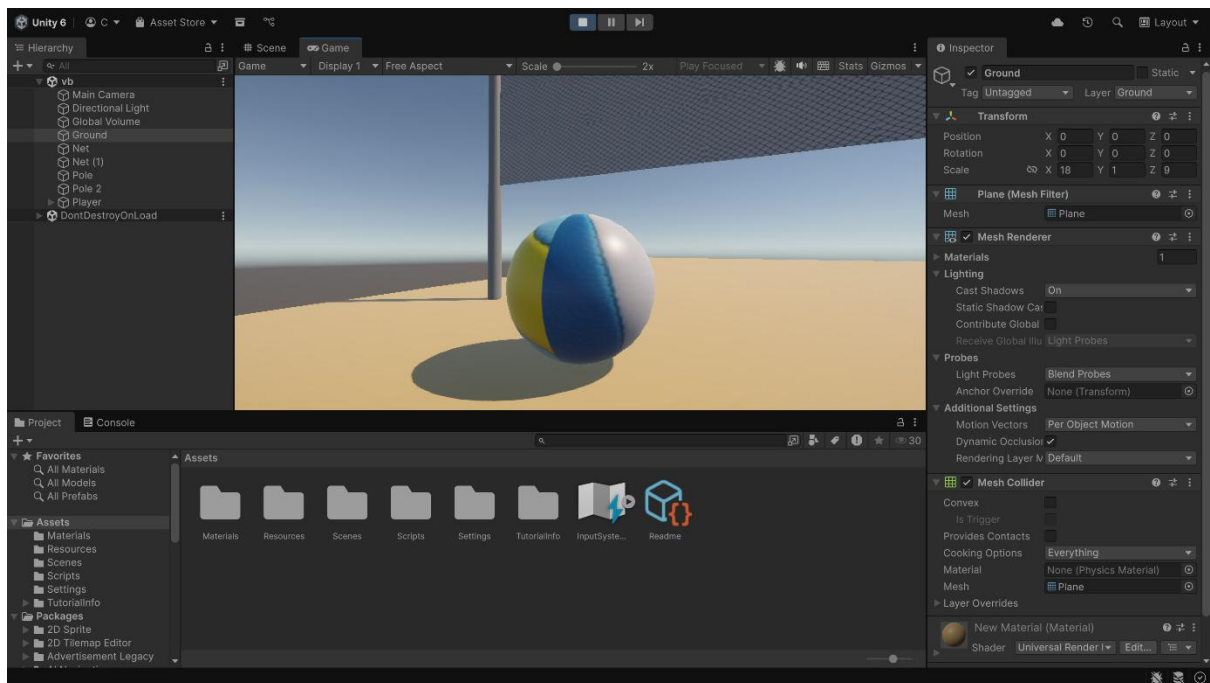


Figure 2: Volleyball moving to other positions

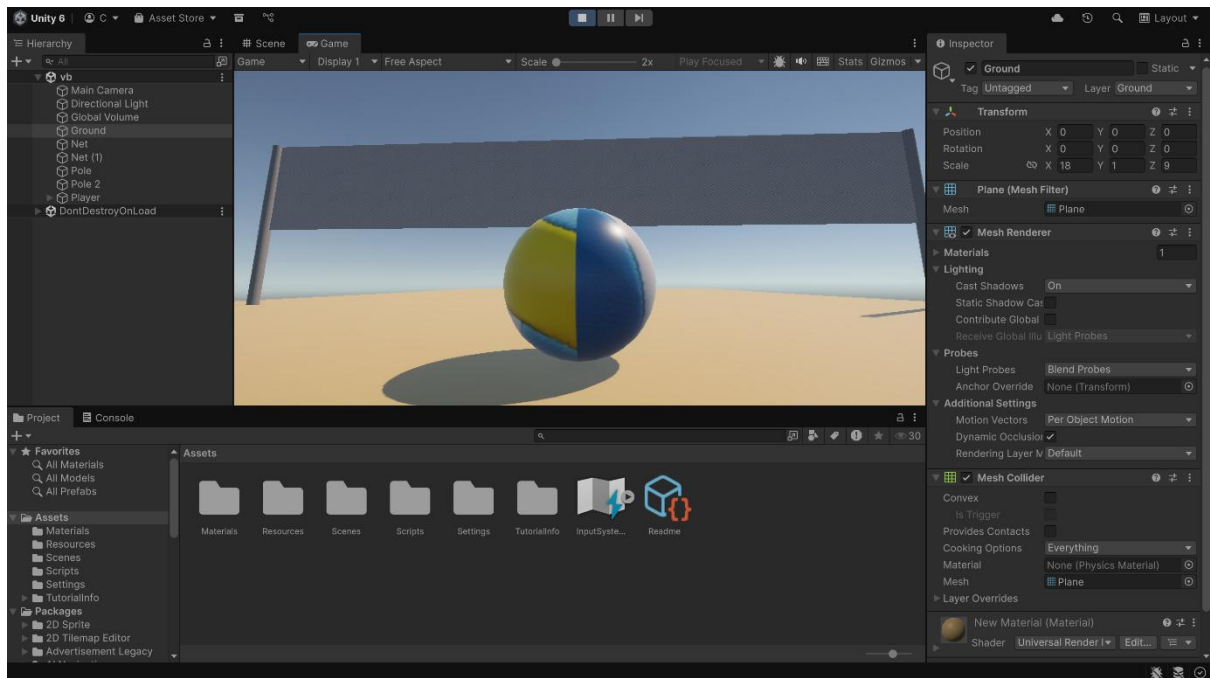


Figure 3: Third person camera

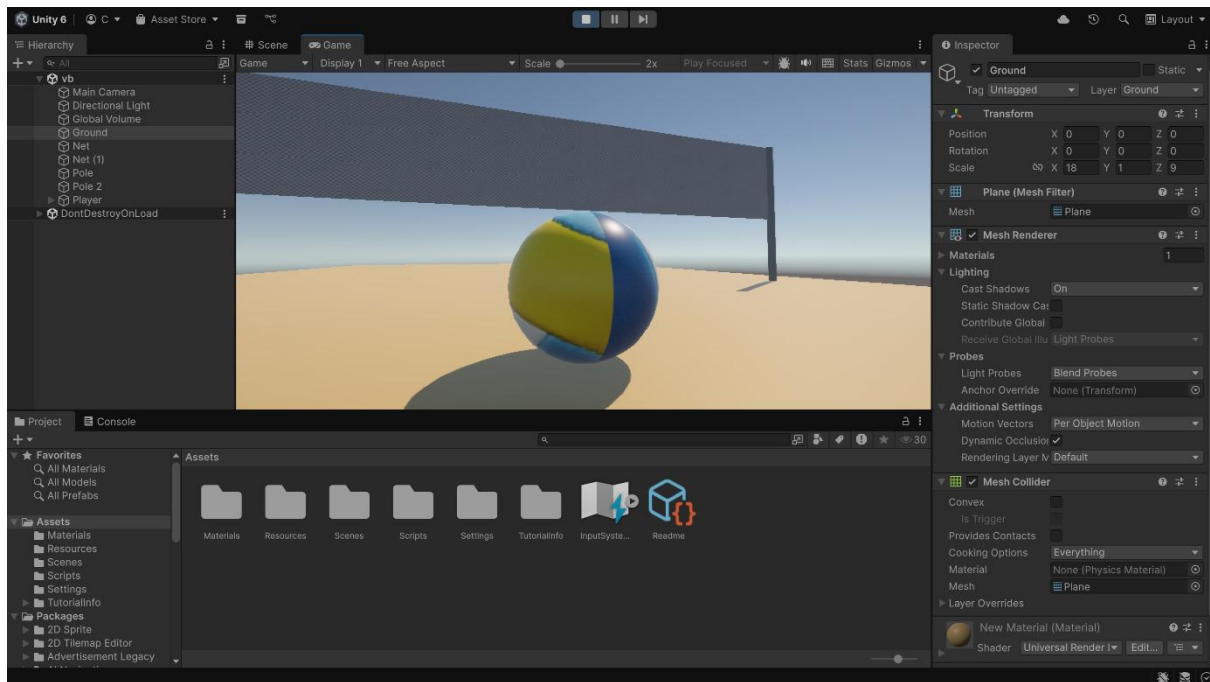


Figure 4: Camera moving as mouse move

References:

1. Unity Technologies, "Unity Scripting API (Unity 6.0 / 6000.3): Physics.CheckSphere; Rigidbody.AddForce; ForceMode.Impulse; Rigidbody.linearVelocity; Physics.SphereCast; Mathf.Clamp; Quaternion.Euler; Quaternion; Vector3.Lerp; MonoBehaviour.LateUpdate," *Unity Documentation*.
2. Unity Technologies, "Unity Manual (Unity 6.0 / 6000.3): Mesh Collider component reference; Cinemachine," *Unity Documentation*.
3. Shutterstock, "Volleyball texture closeup," Asset ID: 1366967630, Apr 12, 2019.
4. NicePNG, "Chain Link Fence PNG Transparent — Net Texture PNG (467×350)," *NicePNG.com*.